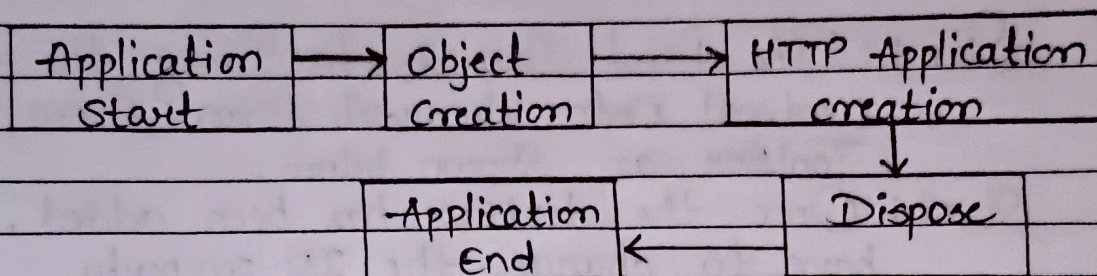
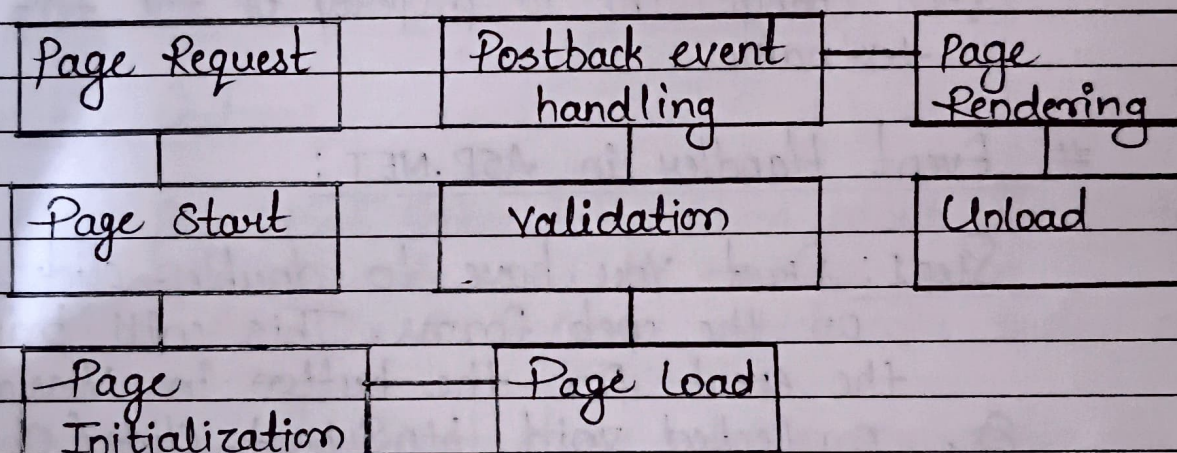


What is ASP.NET Lifecycle ?



What is ASP.NET page lifecycle ?



Adding ASP.NET controls to web forms:

Step 1: Once you perform the above step, you will be able to see your form Designer as shown below. (LABEL CONTROLS)

Step 2: Once the label has been added, follow the following steps.

Step 3: From the properties window, change the name of the text property to name.

Similarly, also change the id property value of the control to lblNAME, By specifying a meaningful ID to controls, it becomes easier to access them during the coding phase.

Text Box:

Step 1: The first Step is to Drag the textBox control onto the web form from the Toolbox as shown below.

Step 2: Once the textBox has been added, you have to change the ID property.

Go to the Properties window by right-clicking on the textBox control and choose properties then change the id property of the textBox to textname.

Event Handler in ASP.NET:

Step 1: First you have to double-click the button on the web forms. This will bring up the event for the button in Visual Studio.

Ex. `protected void btnSubmit_Click (Object sender, EventArgs e) {`
}

The btnSubmit_Click event is automatically added by Visual Studio when you double-click the button in the web forms designer.

Step 2: `protected void btnSubmit_Click (Object sender, EventArgs e) {`

`Response.Write (txtName.Text + "<br>");`
`Response.Write (lstLocation.SelectedItem.Text + "<br>");`

`lblName.Visible = false;`
`txtName.Visible = false;`
`lstLocation.Visible = false;`

`chkC.Visible = false;`
`chkASP.Visible = false;`
`rdMale.Visible = false;`
`rdFemale.Visible = false;`
`btnSubmit.Visible = false;`

}

chap: Your First ASP.NET core Application

Setting the Scene (Problem statement):

- Four key features:

- 1) A home page that shows information about the party.
- 2) A form that can be used to RSVP.
- 3) Validation for the RSVP form, which will display a thank-you page.
- 4) A Summary page that shows who is coming to the party.

Creating a New Project:

Commands {
`dotnet new global::json --sdk-version 6.0.100 --Output PartyInvites`
`dotnet new mvc --no-https --Output PartyInvites --framework net6.0`
`dotnet new sln -o PartyInvites`
`dotnet sln PartyInvites add PartyInvites`

Preparing the Project:

- Setting ports in the launchSetting.json file in the properties folder.

Example:

```
{
  "iisSettings": {
    "windowsAuthentication": false,
    "anonymousAuthentication": true,
    "iisExpress": {
      "applicationUrl": "http://localhost:5000",
      "sslPort": 0
    }
  },
  "profiles": {
    "PartyInvites": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  },
  "IIS Express": {
    "commandName": "IISExpress";
    "launchBrowser": true,
    "environmentVariables": {
      "ASPNETCORE_ENVIRONMENT": "Development"
    }
  }
}
```

- Replacing the contents of the HomeController.cs file in the controller folder,

Ex. Using Microsoft.AspNet.Core.MVC;

```
namespace PartyInvites.Controllers {
  public class HomeController : Controller {
    public IActionResult Index() {
      return View();
    }
  }
}
```

- Replacing the contents of the Index.cshtml file in the Views/Home folder,

Ex. @ {

```
    Layout = null;
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width" />
  <title> Party! </title>
</head>
<body>
  <div>
    <div>
      we're going to have an exciting
      party. <br /> (To do: Sell it better.
      Add pictures or something.)
    </div>
  </div>
</body>
</html>
```

Compiling and Running the project:

commands - dotnet watch

Adding a Data Model:

- The contents of the GuestResponse.cs file in the Models folder.

```
Ex namespace partyInvites.Models {
    public class GuestResponse {
        public string? Name { get; set; }
        public string? Email { get; set; }
        public string? phone { get; set; }
        public bool? WillAttend { get; set; }
    }
}
```

Unit-1

Structure of ASP.NET Core:

Application frameworks			Utility frameworks	
MVC framework	Razor Pages	Blazor	Identity	Entity framework
Platform				
HTTP Server	Middle Ware	URL Routing	Dependency Injection	Configuration & Logging
Caching	Model Binding	Razor	gRPC	SignalR

A) Application Frameworks:-

(1) MVC Framework:

MVC framework is a widely used architecture pattern for building web applications. In the ASP.NET Core MVC framework, the application's logic is divided into three components: the model (data and business logic), the View (User interface), and the controller (handles User requests and updates the model and view accordingly). This framework provides a structured approach to building web applications and supports Separation of Concerns.

(2) Razor Pages:

Razor pages is an alternative to the MVC framework that simplifies the development of page-focused scenarios, with Razor pages, developers can build web applications by organizing code and markup within individual pages. Each Razor page contains both the presentation logic (code) and the corresponding HTML markup, making it easier to create self-contained and cohesive pages. Razor pages are suited for simpler applications or scenarios where the focus is on individual pages rather than the entire application flows.

(3) Blazor:

It is a relatively new framework that enables the development of web applications using C# and .NET instead of traditional JavaScript. Blazor allows developers to build interactive client-side

web UIs Using C# and runs directly in the browser. It leverages webAssembly technology to execute .NET Core in web environment. Blazor offers two hosting models: Blazor webAssembly, where the applications runs entirely on the client-side, and Blazor Server, where the application logic executes on the server while the UI is updated in real-time on the client.

(B) Understanding to Utility Frameworks:-

(1) Entity framework core (EF core):

Entity framework core is Microsoft's object-relational mapping (ORM) framework. It enables developers to work with database using .NET Objects, abstracting away the complexities of interacting with the underlying database systems. EF Core allows you to define your data models using C# or VB.NET classes, and it automatically generates the corresponding database scheme. It provides various features such as querying, data manipulation, and change tracking. EF Core can be used in any .NET application but is commonly used in ASP.NET Core application to access and manipulate relational databases.

(2) ASP.NET Core Identity:

ASP.NET Core Identity is Microsoft's authentication and authorization framework for ASP.NET Core applications. It provides a set of APIs and services to handle user authentication, an

account management and access control with ASP.NET Core Identity, developers can easily implement features like User registration, login, Password management, role-based authorization, and external authentication providers, it simplifies the process of a validating user credentials and managing user identities within an ASP.NET Core application.

Difference Between Application and Utility Frameworks:

(1) Application framework: An Application framework refers to the core set of tools, libraries, and conventions provided by ASP.NET to build web applications. It includes features and patterns for handling HTTP requests and generating responses, a managing routing, providing Session management, and supporting various application architectures like MVC or Razor pages.

- Examples of application frameworks in ASP.NET includes ASP.NET MVC, Razor pages, and Blazor.

(2) Utility framework: An Utility framework, in this context, refers to application additional frameworks or libraries that are not directly responsible for generating HTML content or data but provides an auxiliary functionalities to support ASP.NET applications. These frameworks are often used to handle specific tasks or solve specific problems in an ASP.NET applications.

Example: Utility frameworks in ASP.NET include Entity Framework core (ORM for database access) and ASP.NET core identity (it authentication and authorization framework).

Installing the .NET SDK:

The Visual Studio installer will install the .NET software development kit (SDK), but it may not install the version required then, go to:
<https://dotnet.microsoft.com/download/dotnet-core/6.0>
and download the installer for version 6.0.0 of the .NET SDK, which is the long-term support release at the time of writing.

Run the installer; once the installing is completed, open a new powershell command prompt from the windows start menu and run the command shown in listing 9-1, which displays a list of the installed .NET SDKs.

Listing the installed SDKs:

```
dotnet --list-sdks
```

Installing SQL Server LocalDB:

The database ~~are~~ examples in this book required LocalDB, which is a zero-configuration version of SQL Server that can be installed as part of the SQL Server Express edition, which is available for use without charge from <https://www.microsoft.com/en-in/sql-server/sql-server-downloads>.

Download and run the Express edition installer and select the custom option.

Creating ASP.NET core project:

(1) Creating a New Project:-

```
dotnet new global.json --sdk-version 6.0.100 --output  
FirstProject
```

```
dotnet new MVC --no-https --output FirstProject --  
framework net6.0
```

```
dotnet new sln -o FirstProject
```

```
dotnet sln FirstProject add FirstProject
```

- The first command creates a folder named FirstProject and adds to it a file named global.json, which specifies the version of .NET that the project will use; this ensures you get the expected results when following the examples. The second command creates a new ASP.NET core project. The .NET SDK includes a range of templates for starting new projects, and the MVC template is one of the options available for ASP.NET core applications. This project template creates a project that is configured for the MVC framework, which is one of the application types supported by ASP.NET core.

Creating a Second Action and Views:

- Adding an action method in the HomeController.cs file in the Controllers folder,

Ex. Using Microsoft.AspNet.Mvc;

```
namespace PartyInvites.Controllers {
    public class HomeController : Controller {
        public IActionResult Index() {
            return View();
        }
        public ViewResult RsvpForm() {
            return View();
        }
    }
}
```

- The contents of the RsvpForm.cshtml file in the Views/Home folder,

Ex. @ {

```
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title> RsvpForm </title>
</head>
<body>
    <div> This is the RsvpForm.cshtml View
    </div>
    <a asp-action = "RsvpForm">Rsvp Now </a>
```

```
</body>
</html>
```

Building in form:

- Create a form view in the RsvpForm.cshtml file in the Views/Home folder,

```
@model PartyInvites.Models.GuestResponse
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title> RsvpForm </title>
</head>
<body>
    <form asp-action="RsvpForm" method="post">
        <div>
            <label asp-for="Name"> Your name: </label>
            <input asp-for="Name" />
        </div>
        <div>
            <label asp-for="Email"> Your Email: </label>
            <input asp-for="Email" />
        </div>
        <div>
            <label asp-for="phone"> Your phone: </label>
            <input asp-for="phone" />
        </div>
    </form>
```

```

<div>
  <label asp-for = "WillAttend"> Will you
  attend? </label>
  <Select asp-for = "WillAttend">
    <Option value = ""> Choose an Option </Option>
    <Option value = "true"> Yes, I'll be there.
    </Option>
    <Option value = "false"> No, I can't
    come </Option>
  </select>
</div>
<button type = "submit"> Submit Rsvp </button>
</form>
</body>
</html>

```

Receiving form Data:

(1) A method that responds to HTTPGET requests:
 A Get request is what a browser issues normally each time someone clicks a link. This version of the action will be responsible for displaying the initial blank form where someone first visits Home/RsvpForm.

(2) A method that responds to HTTPPOST requests:
 The form element defined in the method attributes to post, which causes the form data to be sent to be server as a POST request. This version of the action will be responsible for receiving Submitted data and deciding what to do with it.

- Adding a method in the HomeController.cs file in the Controllers folder.

```

Using Microsoft.AspNet.Mvc;
Using PartyInvites.Models;
namespace PartyInvites.Controllers {
  public class HomeController : Controller {
    public IActionResult Index() {
      return View();
    }
    [HttpGet]
    public IActionResult RsvpForm() {
      return View();
    }
    [HttpPost]
    public IActionResult RsvpForm
      (GuestResponse guestResponse) {
      // TODO: store response from guest
      return View();
    }
  }
}

```

Understanding Model Binding:

Adding the Thanks View

Displaying the Responses

Adding Validation and Styling the content

points
to
Referen
book

Chap: Using the Development Tools

Creating ASP.NET Core Project:

(1) Creating a Project Using the command line:

a) Listing the .NET Templates:

`dotnet new --list`

b) Useful ASP.NET Core Project Templates:

- | Names | Description |
|-------------------------|---|
| (1) <u>web</u> | - This template creates a project that is set up with the minimum code and content required for ASP.NET Core development. This is the template I use for most of the chapters in this book. |
| (2) <u>mvc</u> | - This template creates an ASP.NET Core project configured to use the MVC framework. |
| (3) <u>webapp</u> | - This template creates an ASP.NET Core project configured to use Razor pages. |
| (4) <u>blazorserver</u> | - This template creates an ASP.NET Core project configured to use Blazor Server. |
| (5) <u>angular</u> | - This template creates an ASP.NET Core project that contains client-side features using the Angular JavaScript framework. |
| (6) <u>react</u> | - This template creates an ASP.NET Core project that contains client-side features using React JavaScript framework. |

(7) reactredux - This template creates an ASP.NET Core project that contains client-side features using the React JavaScript framework and the popular Redux library.

c) The configuration Item Templates:

Name	Description
------	-------------

- | | |
|--------------------------|---|
| (1) <u>globalization</u> | - This template adds a globalization file to a project, specifying the version of .NET that will be used. |
| (2) <u>sln</u> | - This template creates a solutions file, which is used to group multiple projects and commonly used by Visual Studio. The solution file is a populated with the dotnet sln add commands. |
| (3) <u>gitignore</u> | - This template creates gitignore file that excludes unwanted items from Git source control. |

How to create project by Using .NET command:

Step 1: Creating a New Project,

```
dotnet new globalization --sdk-version 6.0.100 --output  
MySolution/MyProject  
dotnet new web --no-https --output MySolution/  
MyProject --framework net6.0  
dotnet new sln -o MySolution  
dotnet sln MySolution add MySolution/MyProject
```

Step 3: Setting the HTTP Port in the launchSettings.json file in the Properties folder,

```
{
  "iisSettings": {
    "windowsAuthentication": false,
    "anonymousAuthentication": true,
    "iisExpress": {
      "applicationUrl": "http://localhost:5000",
      "sslPort": 0
    }
  },
  "profiles": {
    "MyProject": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Step 3: Building the project,
dotnet build

Step 4: Building and Running the Project,
dotnet run

Managing Packages: 1) Managing NuGet packages:

• Adding a Package to the Example Project:-

command: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 6.0.0

• Listing the Packages in a Project:-

command: dotnet list package

• Removing a Package from the Example Project:-

command: dotnet remove package Microsoft.EntityFrameworkCore.SqlServer

2) Managing tool Packages:

• Installing a Tool Packages:-

command: dotnet tool uninstall --global dotnet-ef
dotnet tool install --global dotnet-ef --
version 6.0.0

• Running a tool Packages command:-

dotnet ef --help

3) Managing client-Side Packages:

• Installing the LibMan Tool package:

command: dotnet tool Uninstall --global Microsoft.
Web.LibraryManager.Cli

dotnet tool install --global Microsoft.Web.Library-
Manager.Cli --version 2.1.0

• Initialization the Example Project:

command: libman init -p cdnjs

- LibMan can download packages from different
repositories.

• Installing the Bootstrap CSS framework:

command: libman install bootstrap@5.1.3 -d
wwwroot/lib/bootstrap

chap: Essential C# Features

• Creating the Example Project for each application.

dotnet new globaljason --sdk-version 6.0.100 --Output
LanguageFeatures

dotnet new web --no-https --Output LanguageFeatures
--framework net6.0

dotnet new sln -o LanguageFeatures

dotnet sln LanguageFeatures add LanguageFeatures

Enabling the MVC framework:

The web project template creates a project that contains a minimal ASP.NET Core Configuration. Sln (solution) file, this means the placeholder content that is added by the MVC template used in chap 3 is not available and that extra steps are required to reach the point where the application can produce useful output. In this section, I make the changes required to set up the MVC framework, which is one of the following application frameworks supported by ASP.NET Core. First, to enable the MVC framework, make the changes shown in enabling MVC to the Program.cs file.

• Enabling MVC in the Program.cs File in the Language Features folder,

```
var builder = webApplication.CreateBuilder(args);  
builder.Services.AddControllersWithViews();  
var app = builder.Build();  
// app.MapGet("/", () => "Hello World!");  
app.MapDefaultControllerRoute();
```

app.Run();

Creating the Application Components:

- The contents of the Product.cs file in the Models folder,

```
namespace LanguageFeatures.Models {  
    public class product {  
        public class String Name { get; set; }  
        public decimal? price { get; set; }  
        public static product[] GetProducts() {  
            product kayak = new product {  
                Name = "kayak", price = 275M };  
            product lifejacket = new product {  
                Name = "Lifejacket", price = 48.95M };  
            return new product[] { kayak, lifejacket, null };  
        }  
    }  
}
```

Creating the controller and View:

- The contents of the HomeController.cs file in the Controllers folder,

```
using Microsoft.AspNetCore.Mvc;  
namespace LanguageFeatures.Controllers {  
    public class HomeController : Controller {  
        public IActionResult Index() {  
            return View(new string[] { "C#", "Language",  
                "Features" });  
        }  
    }  
}
```

```
}  
}  
}
```

- The contents of the Index.cshtml file in the Views/ Home folder,

```
@model IEnumerable<String>  
@ { Layout = null; }  
<!DOCTYPE html>  
<html>  
    <head>  
        <meta name="viewport" content="width=device-width"/>  
        <title>Language Features</title>  
    </head>  
    <body>  
        <ul>  
            @foreach (string s in Model) {  
                <li>@s</li>  
            }  
        </ul>  
    </body>  
</html>
```

Selecting the HTTP Port:

- Setting the HTTP Port in the Language Launch Settings.json file in the Properties folder,

```
{  
    "iis Settings": {  

```

```

"WindowsAuthentication": false,
"anonymousAuthentication": true,
"IIS Express": {
  "applicationUrl": "https://localhost:5000",
  "sslPort": 0 }
}

```

```

"profiles": {
  "LanguageFeatures": {
    "commandName": "Project",
    "dotnetRunMessages": true,
    "launchBrowser": true,
    "applicationUrl": "https://localhost:5000",
    "environmentVariables": {
      "ASPNETCORE_ENVIRONMENT": "Development"
    }
  }
}

```

```

"IIS Express": {
  "commandName": "IIS Express",
  "launchBrowser": true,
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Development"
  }
}
}

```

Running the Example Application:

command: dotnet run

Understanding Global Using Statements:

- Adding a Using Statement in the HomeController.cs File in the Controllers folder,

```

using Microsoft.AspNetCore.Mvc;
using LanguageFeatures.Models;
namespace LanguageFeatures.Controllers {
  public class HomeController: Controller {
    public IActionResult Index() {
      Product[] products = Product.GetProducts();
      return View(new string[] { products[0].Name });
    }
  }
}

```

- The contents of the GlobalUsings.cs file in the Language-Features folder,

```

global using LanguageFeatures.Controllers;
global using Microsoft.AspNetCore.Mvc;

```

- Removing Using Statements in the HomeController.cs File in the Controllers folder,

```

namespace LanguageFeatures.Controllers {
  public class HomeController: Controller {
    public IActionResult Index() {
      Product[] products = Product.GetProducts();
      return View(new string[] { products[0].Name });
    }
  }
}

```

Understanding Implicit Using Statements:

- (i) System
 (ii) System.Collections.Generic
 (iii) System.IO
 (iv) System.Linq
 (v) System.Net.Http
 (vi) System.Net.Http.Json
 (vii) System.Threading
 (viii) System.Threading.Tasks
 (ix) Microsoft.AspNetCore.Builder
 (x) Microsoft.AspNetCore.Hosting
 (xi) Microsoft.AspNetCore.Http
 (xii) Microsoft.AspNetCore.Routing
 (xiii) Microsoft.AspNetCore.Configuration
 (xiv) Microsoft.Extensions.Configuration
 (xv) Microsoft.Extensions.DependencyInjection
 (xvi) Microsoft.Extensions.Hosting
 (xvii) Microsoft.Extensions.Logging

• Checking for Null Values:

- (1) Changing Type in the HomeController.cs file in the Controller folder,

```
namespace LanguageFeatures.Controllers {
    public class HomeController: Controller {
        public IActionResult Index() {
            Product[] products = Product.GetProducts();
            return View(new string[] { products[0].Name });
        }
    }
}
```

- (2) Guarding Against a new null Value in the HomeController.cs file in the Controllers folder,

```
namespace LanguageFeatures.Controllers {
    public class HomeController: Controller {
        public IActionResult Index() {
            Product p = products[0];
            Product[] products = Product.GetProducts();
            string val;
            if (p != null) {
                val = p.Name;
            }
            else {
                val = "No Value";
            }
            return View(new string[] { val });
        }
    }
}
```

Chap: Testing ASP.NET Core Applications

• Creating the Example Project:

```
dotnet new globaljson --sdk-version 6.0.100 --Output
Testing/SimpleApp
dotnet new web --no-https --Output Testing/SimpleApp
--framework net 6.0
dotnet new sln -o testing
dotnet sln Testing add Testing/SimpleApp
```

• Opening the Project

Selecting HTTP port

Enabling MVC

Refer to
reference book

• Creating a Unit Test Project:

- The Unit test Project tools:-

- | Name | Description |
|-------------------|---|
| (a) <u>mstest</u> | - This template creates a project configured for the MS Test framework, which is produced by Microsoft. |
| (b) <u>nunit</u> | - This template creates a project configured for the NUnit framework. |
| (c) <u>xunit</u> | - This template creates a project configured for the xUnit framework. |

• Creating the Unit Test Project:

```
dotnet new xunit -o SimpleApp.Tests --framework
net 6.0
```

```
dotnet sln add SimpleApp.Tests
dotnet add SimpleApp.Tests reference SimpleApp
```

• Removing the Default Test class:

- Removing the default Test class File:

Remove - Item SimpleApp.Tests/Unit Test1.cs

• Commonly Used xUnit.net Assert Methods:

Name	Description
(1) <u>Equal (expected, result)</u>	This method asserts that the result is equal to the expected Outcome. There are Overloaded versions of this method for comparing different types and for comparing collections. There is also a version of this method that accepts an additional arguments of an Object that implements the <code>IEqualityCompare<T></code> interface for comparing Objects.
(2) <u>Not-Equal (expected, result)</u>	This method asserts that the result is not equal to the expected Outcome.
(3) <u>True (result)</u>	This method asserts that the result is true.
(4) <u>false (result)</u>	This method asserts that the result is false.

- (5) IsType(expected, result) This method asserts that the result is of a Specific type.
- (6) IsNotType(expected, result) This method asserts that the result is not a Specific type.
- (7) IsNull(result) This method asserts that the result is Null.
- (8) IsNotNull(result) This method asserts that the result is not Null.
- (9) InRange(result, low, high) This method asserts that the result falls between low and high.
- (10) NotInRange(result, low, high) This method asserts that the result falls Outside low and high.
- (11) Throws(expectation, expression) This method asserts that the Specified expectation expression throws a Specific exception type.

• Example: `Assert.Equal("Name New", p.Name);`

• Running Tests from the command line:

`dotnet test`

Refer to
reference
book

• correcting the Unit Test:

Refer to
Ref. Book #

#

•

th

{